COSC204
Computer Systems

Introduction

And

Bits and Bytes

Overview

- This lecture
  - Class outline
  - Bits, bytes, and characters
  - This lecture should be mostly revision
  - Source: Chapter 10 (11th edition)
- Next lecture
  - Combinatorial Logic

Instructors & Support People

- Andrew Trotman (course coordinator)
  - Office: 122A, Owheo
  - Email: andrew.trotman@otago.ac.nz
- Zhiyi Huang
  - Office: 126, Owheo
  - Email: zhiyi.huang@otago.ac.nz
- Teaching Fellow (in the labs):
  - Chris Edwards
- Kaiāwhina (Māori and Pacific Support)
  - Steven Mills, and Karen Gray
- Disability Support
  - Contact Disability Information and Support

Learning Outcomes

- By completion of this paper students are expected to:
  - Understand the principals of digital electronics (transistors to logic gates to the CPU and RAM)
  - Understand how programs are executed on computer hardware
  - Demonstrate an ability to read and write assembly language programs
  - Be able to construct programs in C and understand its role as a systems programming language
  - Understand the fundamental principles and structure of an operating system
  - Understand core operating systems algorithms such as process scheduling and resource allocation and management
  - Be aware of the importance of efficient use of resources from computational, economic, and environmental viewpoints

Textbooks

- Textbooks (Recommended)
- Architecture / C
  - Stallings, W. Computer Organization and Architecture, 11th Edition
  - Kernighan B. W. & Ritchie, D. C Programming Language, 2nd Edition
- Operating Systems
  - Bryant, R. & O'Hallaron, D. Computer Systems: A Programmers Perspective, 3rd edition, 2021
  - Siberschatz, A. et al. Operating System Concepts (10th edition), Wiley

Class Details

- Two lectures per week
- Two labs per week
  - An integral part of the course (and marked)
  - Yes, there are labs this week
- Consultation times
  - Just “drop in” to our offices

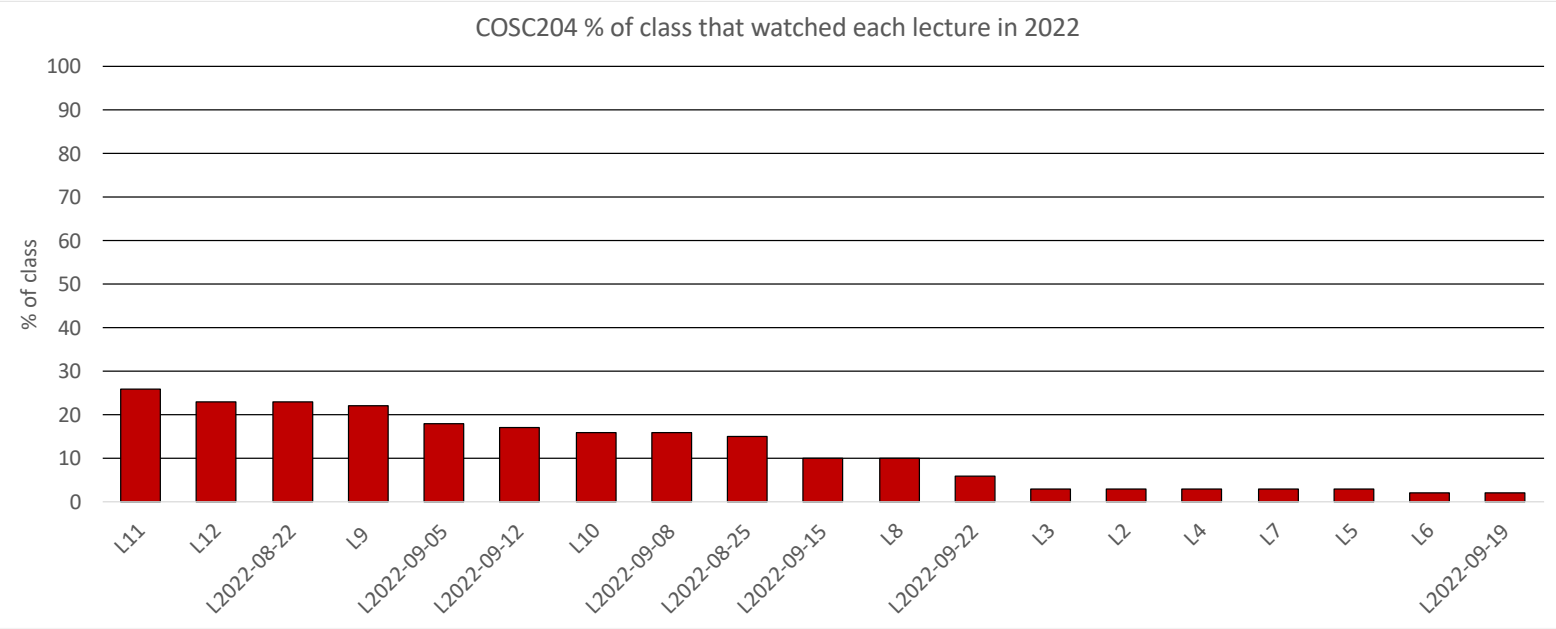
Class Details (cont)

- 18 point paper:
  - "As a guide, based on an 18-point paper, this usually works out to a minimum of 14 hours work per paper per week. Work is made up of formal contact time (lectures, tutorials, laboratories, etc.) and independent study (studying, revision, assignments, reading, etc.)." https://www.otago.ac.nz/study/planning/workload.html
  - 6 contact hours/week * 13 weeks = 78 hours of 180 total
  - How many non-contact hours should you put in at home?

- Please check your email regularly

Online Resources

- Aoroa
  - Lecture notes / lab sheets / Lecture recordings
  - Links to extra resources
  - Assignment sheets



Assessment

- Internal
  - Labs (10%)
    - There are 22 assessed labs, each worth 0.5% (capped at 10%)
  - Architecture / C (25%)
    - Test (closed book)
      - 10%: Hardware (held during Lab 5, week of 27 July)
    - Assignment
      - 15%: Programming in 6809 assembly language (already online)
      - Due: Marked during Lab 9 (week of 10 August)
  - Operating Systems (25%)
    - Test (closed book)
      - 10%: C (held during Lab 15, week of 7 September)
    - Assignment
      - 15%: C Programming, OS File System (released week 8, due week 12)
- Final Exam
  - 40%: 2-hour multi-choice exam
- Your ***combined*** tests + exam score ***must*** exceed 40%

Academic Integrity and Academic Misconduct

- Academic integrity means being honest in your studying and assessments. It is the basis for ethical decision-making and behaviour in an academic context. Academic integrity is informed by the values of honesty, trust, responsibility, fairness, respect and courage. Students are expected to be aware of, and act in accordance with, the University's Academic Integrity Policy.

- Academic Misconduct, such as plagiarism or cheating, is a breach of Academic Integrity and is taken very seriously by the University. Types of misconduct include plagiarism, copying, unauthorised collaboration, submitting work written by someone else (including from a file sharing website, text generation software, or purchased work) taking unauthorised material into a test or exam, impersonation, and assisting someone else's misconduct. A more extensive list of the types of academic misconduct and associated processes and penalties is available in the University's Student Academic Misconduct Procedures.

Academic Integrity and Academic Misconduct

- It is your responsibility to be aware of and use acceptable academic practices when completing your assessments. To access the information in the Academic Integrity Policy and learn more, please visit the University's Academic Integrity website at www.otago.ac.nz/study/academicintegrity, or ask at the Student Learning Centre (CEDI) or the Library, or seek advice from your paper co-ordinator.
- For further information:
- **Academic Integrity Policy**
  - http://www.otago.ac.nz/administration/policies/otago116838.html
- **Student Academic Misconduct Procedures**
  - http://www.otago.ac.nz/administration/policies/otago116850.html

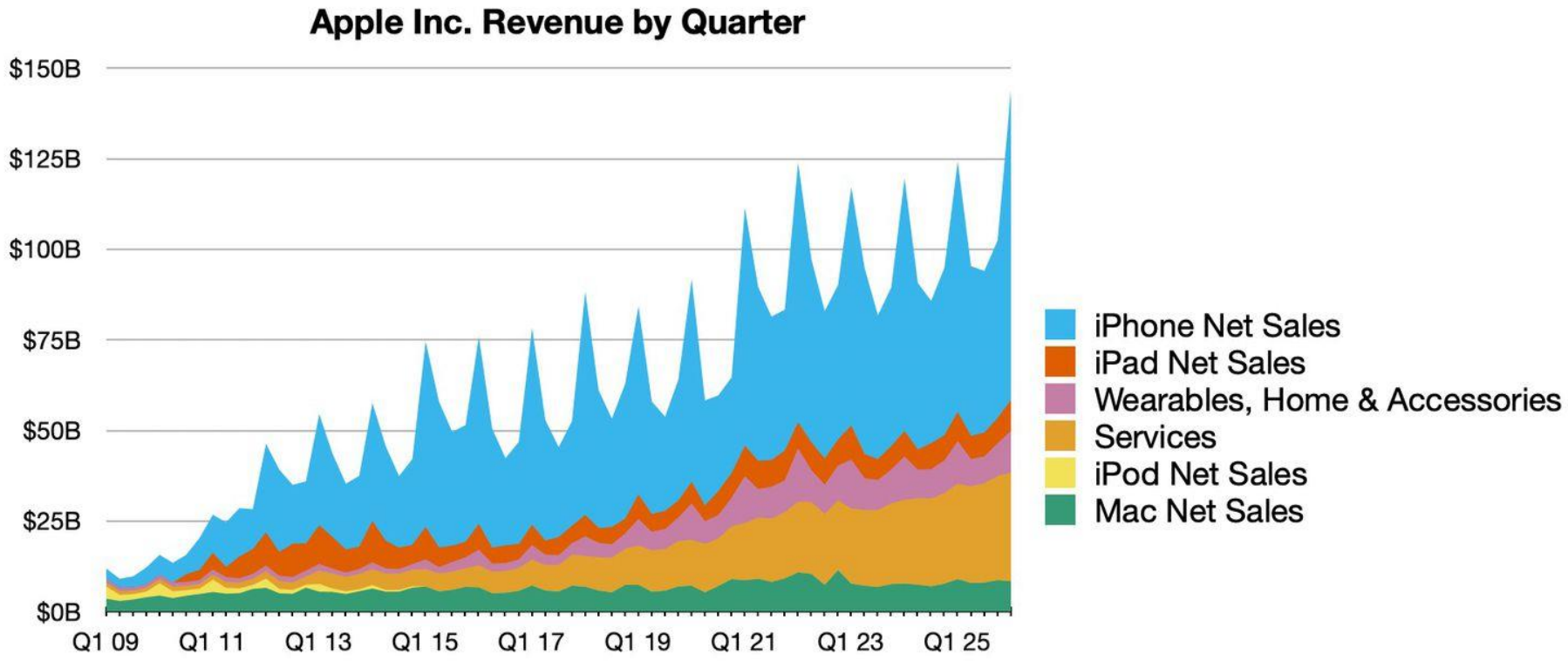
Course Outline

- Introduction to Computer Architecture
- Digital electronics (transistors, LEDs, resistors, logic gates)
- Digital Logic, the CPU, and Computer Architecture
- Machine Code, 6809 Assembly
- Fundamentals of C programming
- Pointers and Memory Management in C
- C as a Systems Programming Language
- Operating Systems Fundamentals
- Processes, Threads, and Scheduling
- Memory and Storage
- File Systems and I/O
- Current Trends in Computer Systems

Why Study Comp. Systems

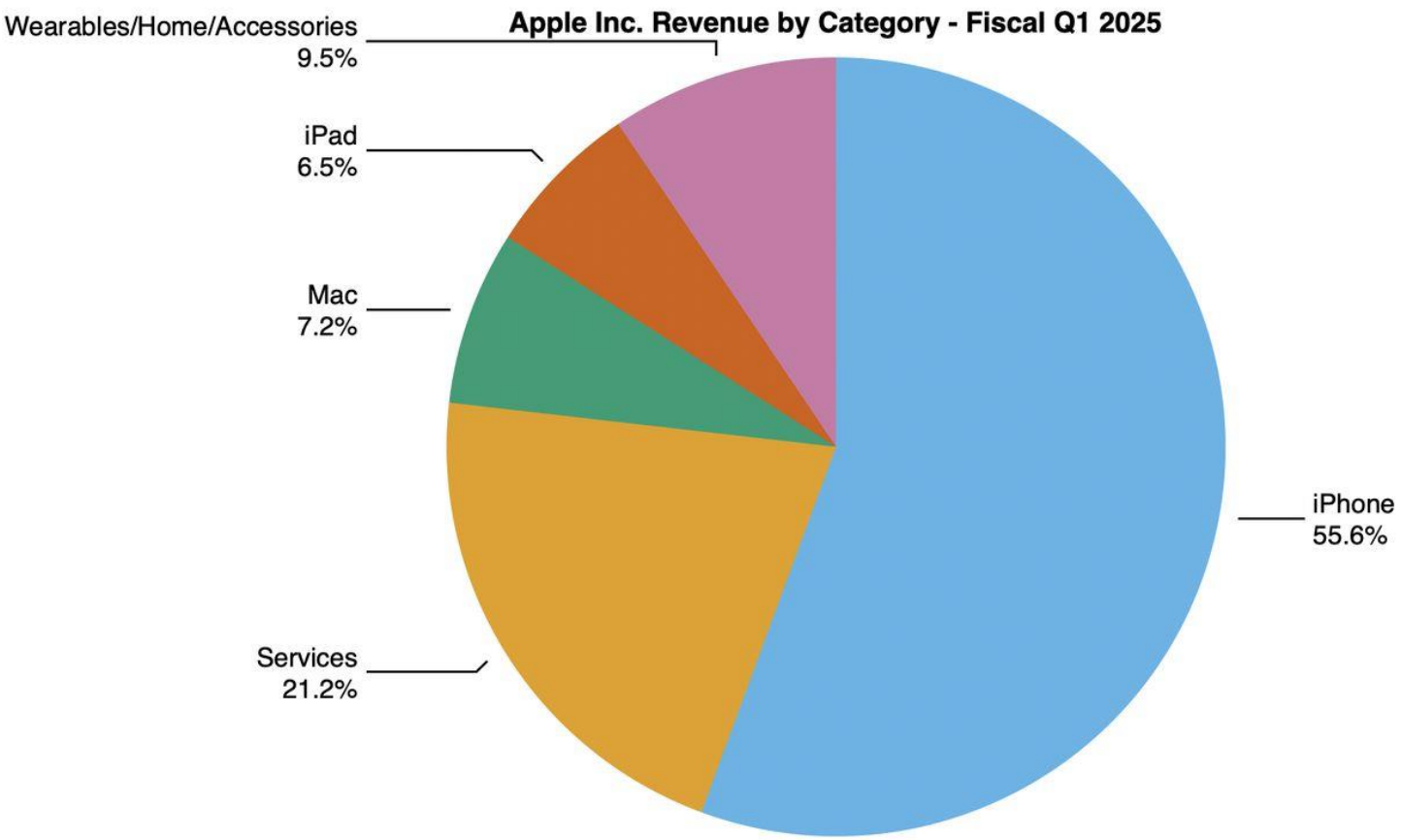
- Understand the underlying hardware
  - So that we know what our programs can and cannot do
    - We often ignore architecture when doing high-level programming (in Java, C, etc.)
- Understand the underlying software
  - For assembly language programming
  - To take advantage of hardware advances (e.g. AVX512)
  - For writing system software such as device drivers
  - ***To understand what our programs mean!***

What is a Computer?



https://www.macrumors.com/2026/01/29/apple-1q-2026-earnings/

Desktop Computing Minority



https://www.macrumors.com/2025/01/30/apple-1q-2025-earnings/

What is a Computer?



What is Data?

- Data – a fact (information)
  - Data corresponds to discrete facts about phenomena from which we gain information about the world
- The concept of ***value*** is fundamental to data
  - Examples: 25, $256.50, April, “This is a sentence”, colours, etc.
- Values are abstract, they are interpretations of data
  - And the same value may be written (or stored) in different ways
    - Examples: 12, twelve, XII, 1100, • ••, • - - - - • • - - -

How Computers Represent Data

- The digital computer is binary

- Everything is stored in one of two states: true, false
  - Which can be thought of as 1 and 0
  - Or a switch which is on or off (voltage, no voltage)



or



  - In some chips it's a fuse that is "open" or "blown"
- An instance of a state is called a ***bit*** (or binary digit)

How Computers Represent Data

- Values are represented as sequences bits: 1000001



- What does this sequence mean?
- How does the computer know what any particular sequence of bits means?
  - The computer doesn't know – ***you*** know
  - It might be:
    - 65 (in decimal)
    - 'A' (in ASCII)
    - Or anything else ***you*** want it to mean!

Computer Memory

- Memory is just like the switchboard in your home, each switch has a number. They're all there all the time and each has a state (***on*** or ***off***). You change the state by flipping the switch

- In the computer each switch has:
  - An address or location (the switch number)
  - A value (the state, on or off)

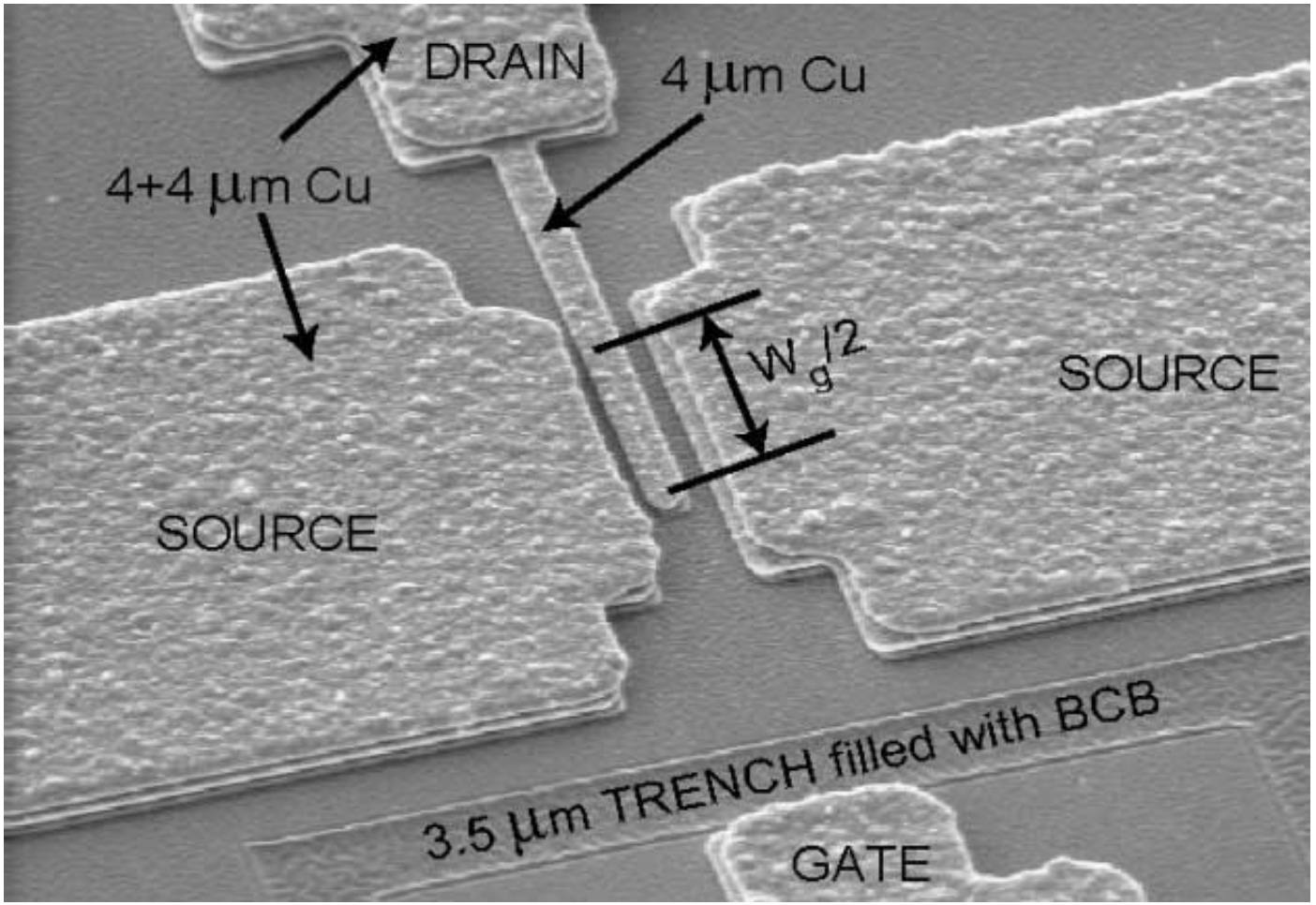
- Computer languages let us name some of the locations: it's easier than remembering numbers

Bank of Switches



https://www.amazon.co.uk/Chrome-Customised-Kitchen-Switch-Switches/dp/B00WPUPD1E

Transistor: the Computer Switch

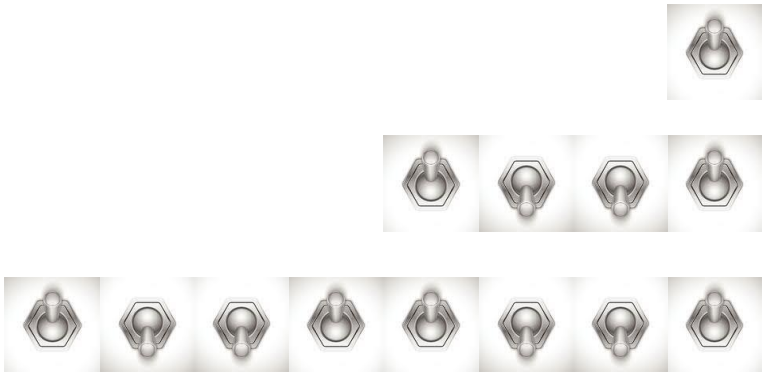


https://www.researchgate.net/figure/Scanning-electron-microscopy-image-of-two-levels-of-copper-electroplated-on-the_fig2_3254792

Bits, Nybbles, Bytes

- The smallest unit of storage is a ***bit*** (1 or 0)
- For convenience bits are grouped together into larger units. A ***nybble*** (***nibble***, or ***nyble***) is 4 consecutive bits. One row on the example “Bank of switches”
- A ***byte*** is 8 consecutive bits (2 nybbles)
- For convenience it is bytes that are given addresses in the (modern) computer (bits and nybbles are too small to work with most of the time).

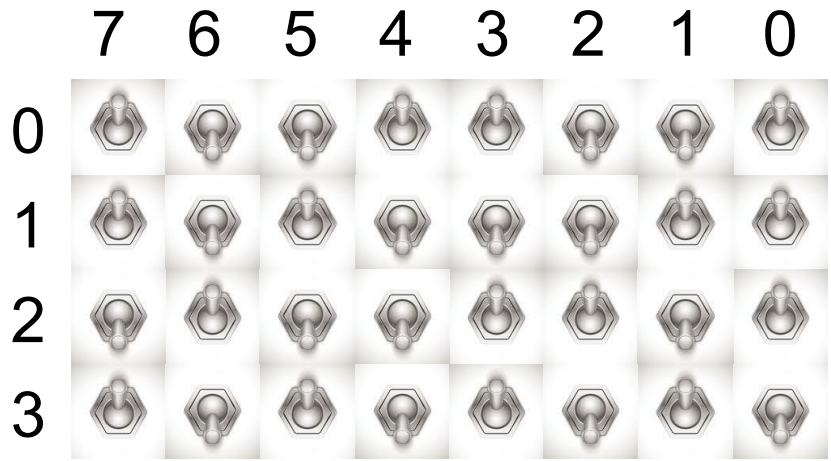
Bits, Nybbles, Bytes



1 bit = 1 switch

1 nybble = 4 switches

1 byte = 8 switches



Computer memory is a switchbox of transistors

A Word of Memory

- The ***word*** is the number of bits the CPU uses internally. It varies from manufacturer to manufacturer and from CPU to CPU. These days it is usually 64 bits

- Bits — Computer
  - 8 — Atari 800, Commodore 64, Apple][
  - 16 — IBM PC, Macintosh, DEC VAX
  - 18 — DEC PDP 15
  - 32 — IBM 370, early Android and iOS phones
  - 36 — Honeywell, UNIVAC
  - 64 — Cray-1, Mac, PC, iPhone, etc.

Characters

- A ***character*** is a written symbol. They are (for the Roman alphabet (English)) represented as a single byte. For other languages they may be 2 bytes or more
- Characters include:
  - UPPER CASE LETTERS A C X . . .
  - lower case letters b g k . . .
  - Single digits 3 1 9 . . .
  - Special characters ! < = . . .
- We can join these together in sequences to make (human readable) numbers and words

Characters

- `uint8_t ch;`

- `ch` is a variable name (an identifier). It is used to label a particular part of the computer's memory where a ***byte*** value is stored. The ***byte*** is a sequence of 8 ***bits***, an 8-bit integer [0..255].
- During compilation, the name is assigned an address in the computer's memory – the location where the data is stored. The meaning of that data depends on how a human interprets it. It might be a small integer, or a character, or a colour, etc.

Displaying an ASCII Character

- How does the computer know that `ch` is an '`A`' and not '`a`' or '`3`' or anything else?
  - It doesn't!

1.The computer uses `ch` as an integer index into a pre-existing table, e.g.

```
00010000001010000100010010000010100000101111111010000010100000010
```

- Or, for clarity, an array of bytes like this:

```
uint8_t bits[65] = {16, 40, 68, 130, 130, 254, 130, 130}
```

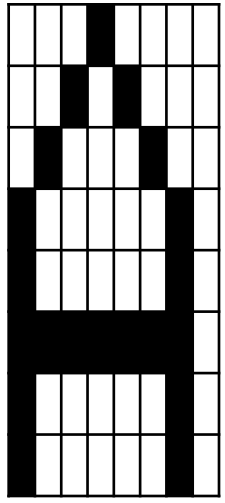
2. The computer screen is made up of thousands of little dots called pixels. They are in a rectangular grid, like a table.

Displaying an ASCII Character

00010000001010000100010010000010100000101111111010000010100000010

00010000 00101000 01000100 10000010 10000010 11111110 10000010 10000010





When the computer is asked to “print” a character it turns on and off some of those pixels. ***We*** then interpret the drawing.

We impose meaning: ***We*** give it a value

ASCII Character Codes

- There are several standard tables that describe what to draw
  - Fonts describe how to draw them

- ***ASCII*** (the American Standard Code for Information Interchange) describes what should be drawn for Roman (English-like) alphabets
  - For example;

```
A     1000001 (65)
a     1100001 (97)
9     0111001 (57)
```

- But there are only a few letters, numbers, and punctuation marks. The remaining ASCII codes are non-printing and have other meaning (such as line feed, form feed, tab, etc.).

ASCII Character Codes

- ASCII characters are stored using 7-bits
  - There are, therefore, 128 (2^7) possible characters. They are usually stored in memory as a ***byte*** with the 8th bit (the high bit) set to 0

- For sorting purposes characters are compared on their numeric value (called the collating sequence).
  - 'A' is before 'Z' but 'a' is after 'Z'!

Unicode

- What about all the non-Roman characters?
  - Greek, Arabic, Chinese, Hebrew, Japanese, Thai, etc.
  - Astrology symbols etc.
- What about characters that represent ideas, concepts, phrases?
  - Emoji, etc.
- Unicode
  - Developed by The Unicode Consortium
  - Coordinated with ISO/IEC 10646
  - A 21-bit code with 159,801 characters from 172 scripts
  - Unicode maps from character numbers (code points) into glyphs (graphical representations)
  - Some (many) are reserved
  - https://en.wikipedia.org/wiki/List_of_Unicode_characters#Basic_Latin

Homework

- Many of the lectures will have a homework slide
  - These are for your entertainment
  - I do ***not*** mark the homework
  - I do ***not*** supply model answers

Homework

- How are character strings (e.g. “hello world”) stored in a computer?
- Is this different between different programming languages (for example; C and Java)?

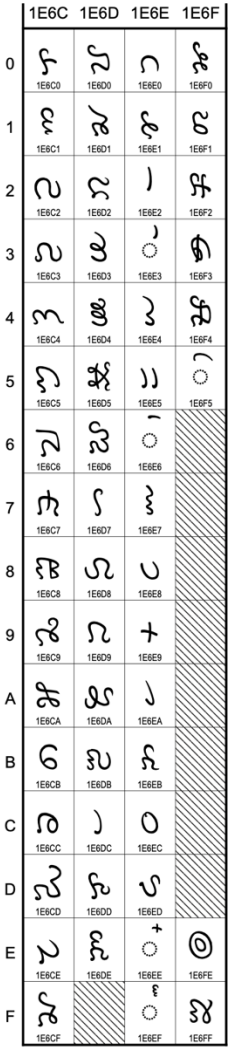
ASCII

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	|
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

https://sites.google.com/site/sblasa2computing/f451---computing-fundamentals/1-3-data-repressenation/1-3-5-character-sets

Unicode

1E6C0 **Tai Yo** **1E6FF**



Letters

1E6C0 TAI YO LETTER LOW KO
1E6C1 TAI YO LETTER HIGH KO
1E6C2 TAI YO LETTER LOW KHO
1E6C3 TAI YO LETTER HIGH KHO
1E6C4 TAI YO LETTER GO
1E6C5 TAI YO LETTER NGO
1E6C6 TAI YO LETTER CO
1E6C7 TAI YO LETTER LOW XO
1E6C8 TAI YO LETTER HIGH XO
1E6C9 TAI YO LETTER LOW NYO
1E6CA TAI YO LETTER HIGH NYO
1E6CB TAI YO LETTER DO
1E6CC TAI YO LETTER LOW TO
1E6CD TAI YO LETTER HIGH TO
1E6CE TAI YO LETTER THO
1E6CF TAI YO LETTER NO
1E6D0 TAI YO LETTER BO
1E6D1 TAI YO LETTER LOW PO
1E6D2 TAI YO LETTER HIGH PO
1E6D3 TAI YO LETTER PHO
1E6D4 TAI YO LETTER LOW FO
1E6D5 TAI YO LETTER HIGH FO
1E6D6 TAI YO LETTER MO
1E6D7 TAI YO LETTER YO
1E6D8 TAI YO LETTER LO
1E6D9 TAI YO LETTER VO
1E6DA TAI YO LETTER LOW HO
1E6DB TAI YO LETTER HIGH HO
1E6DC TAI YO LETTER QO
1E6DD TAI YO LETTER LOW KVO
1E6DE TAI YO LETTER HIGH KVO

Vowel letters and rhyme signs

1E6E0 TAI YO LETTER AA
1E6E1 TAI YO LETTER I
1E6E2 TAI YO LETTER UE
1E6E3 TAI YO SIGN UE
1E6E4 TAI YO LETTER U
1E6E5 TAI YO LETTER AE
1E6E6 TAI YO SIGN AU
1E6E7 TAI YO LETTER O
1E6E8 TAI YO LETTER E
1E6E9 TAI YO LETTER IA
1E6EA TAI YO LETTER UEA
1E6EB TAI YO LETTER UA
1E6EC TAI YO LETTER OO
1E6ED TAI YO LETTER AUE

Finals

1E6EE TAI YO SIGN AY
1E6EF TAI YO SIGN ANG
1E6F0 TAI YO LETTER AN
1E6F1 TAI YO LETTER AM
1E6F2 TAI YO LETTER AK
1E6F3 TAI YO LETTER AT
1E6F4 TAI YO LETTER AP
1E6F5 TAI YO SIGN OM

Symbols

1E6FE TAI YO SYMBOL MUEANG
1E6FF TAI YO XAM LAI

The Unicode Standard, Version 17.0, Copyright © 1991-2025 Unicode, Inc. All rights reserved.

1782